

UNIVERSIDAD PERUANA LOS ANDES



CURSO: DISEÑO DE EXPERIENCIA DE USUARIO UX&UI

***DOCENTE: Raul Enrique
Fernandez Bejarano***

ALUMNO:

Ñahuincopa Huamani Lian Renzo

CICLO: VI

Semana 13

***HUANCAYO-
2025***

Análisis de rendimiento con SQL Profiler y Extended Events

Definición:

El análisis de rendimiento consiste en capturar y revisar eventos que ocurren dentro del motor de SQL Server, como ejecución de consultas, esperas, bloqueos o consumo excesivo de recursos.

Las dos herramientas principales son:

SQL Server Profiler

- Herramienta gráfica tradicional.
- Permite capturar *trazas* (traces) en tiempo real.
- No se recomienda en producción porque consume bastantes recursos.

Extended Events

- Reemplazo moderno y liviano del Profiler.
- Consume muchos menos recursos.
- Permite capturar eventos detallados del motor (esperas, bloqueos, deadlocks, CPU elevada, etc.).

Ejemplo (crear sesión de Extended Events):

```
CREATE EVENT SESSION Monitor_CPU ON SERVER
```

```
ADD EVENT sqlserver.rpc_completed(
```

```
    ACTION(sqlserver.sql_text)
```

```
    WHERE duration > 1000
```

```
)
```

```
ADD TARGET package0.event_file
```

```
(SET filename = 'C:\XE\MonitorCPU.xel');
```

GO

```
ALTER EVENT SESSION Monitor_CPU ON SERVER STATE = START;
```

Estadísticas e índices (creación, fragmentación, mantenimiento)

Definición:

Estadísticas

- Datos internos que usa SQL Server para estimar cuántas filas devolverá una consulta.
- Importantes para elegir un buen plan de ejecución.
- Deben mantenerse actualizadas.

Índices

- Estructuras que aceleran la búsqueda de datos.
- Pueden fragmentarse con el tiempo (especialmente los índices tipo *B-tree*).
- Deben reorganizarse o reconstruirse según el nivel de fragmentación.

• Ejemplo: Crear un índice

```
CREATE INDEX IX_Ventas_Fecha  
ON Ventas(FechaVenta);
```

• Ejemplo: Actualizar estadísticas

```
UPDATE STATISTICS Ventas;
```

• **Ejemplo: Ver fragmentación**

```
SELECT
```

```
    index_id,
```

```
    avg_fragmentation_in_percent
```

```
FROM sys.dm_db_index_physical_stats(DB_ID(), OBJECT_ID('Ventas'), NULL, NULL,  
'SAMPLED');
```

Administración de transacciones y bloqueos

☒ Definición:

Las transacciones garantizan que las operaciones se ejecuten completas o ninguna.
Los bloqueos aseguran que los datos permanezcan consistentes cuando varios usuarios acceden al mismo recurso.

Tipos de bloqueo comunes:

- **Shared (S)**: para lectura.
- **Exclusive (X)**: para escritura.
- **Update (U)**: previo a un X.
- **Intent Locks**: indican intención de bloquear.

Con mala gestión pueden ocurrir:

- **Deadlocks** (dos sesiones esperando recursos entre sí)
- **Bloqueos largos**

- Escalación de bloqueos

Ejemplo de transacción con manejo explícito:

```
BEGIN TRAN;
```

```
UPDATE Productos
```

```
SET Stock = Stock - 1
```

```
WHERE IdProducto = 10;
```

```
COMMIT TRAN;
```

Análisis de planes de ejecución

☒ Definición:

Un plan de ejecución muestra cómo SQL Server decide obtener los datos: uso de índices, operaciones de join, estimaciones de filas, costos, etc.

Se utiliza para:

- Detectar *table scans*, *key lookups*, operadores costosos.
- Identificar índices faltantes.
- Detectar malos estimadores de cardinalidad.

Ejemplo: Mostrar plan estimado

```
SET SHOWPLAN_XML ON;
```

```
GO
```

```
SELECT * FROM Ventas WHERE IdCliente = 1500;
```

GO

SET SHOWPLAN_XML OFF;

Optimización de consultas T-SQL

Definición:

Consiste en mejorar consultas para que utilicen menos recursos y se ejecuten más rápido.

Incluye:

- Reescribir consultas.
- Usar índices adecuados.
- Evitar funciones en columnas indexadas.
- Reducir subconsultas innecesarias.
- Usar JOINS correctamente.
- Controlar operaciones costosas (ORDER BY, GROUP BY, DISTINCT).

Ejemplo de consulta no óptima:

SELECT *

FROM Ventas

WHERE YEAR(FechaVenta) = 2024;

Optimizada:

SELECT *

FROM Ventas

```
WHERE FechaVenta >= '2024-01-01'
```

```
AND FechaVenta < '2025-01-01';
```

Control de recursos con Resource Governor

Definición:

Resource Governor permite limitar el uso de CPU y memoria por grupos de trabajo en SQL Server.

Sirve para evitar que una aplicación o usuario consuma todos los recursos del servidor.

Puede controlar:

- CPU
- Memoria
- Requests en cola

Ejemplo: Crear un pool y grupo de recursos

```
CREATE RESOURCE POOL Pool_ConsultasLentas
```

```
WITH (MAX_CPU_PERCENT = 20);
```

```
CREATE WORKLOAD GROUP GrupoLento
```

```
USING Pool_ConsultasLentas;
```

```
ALTER RESOURCE GOVERNOR RECONFIGURE;
```